Canvas AGS verification service: build a production-ready package with OAuth2 token flows, score-sync loops, and LTI signature validation

Computer
Canvas AGS lineitem sync engine: build an idempotent sync worker with retry queues, batch score updates, and automated audit logging
Computer
Walk me through the ags mjs score sync implementation
Show me how service mjs handles the http verification layer
Explain how the core mjs jwks caching and verification works

The reference package now has an additional hardening extension for retry classification, ordered batch dispatch, and a PostgreSQL queue/audit schema scaffold. Its seven local tests passed, but the combined implementation is not yet production-ready: durable database coordination, tenant permissions, network controls, and live LMS acceptance tests remain unfinished.

Below is how the existing three modules work and where the hardening extension fits.

1. `ags.mjs` score synchronization

Client responsibilities

`agsClient()` wraps four operations:

Method	Responsibility
`discover()`	Find the activity's existing gradebook column
`create()`	Create a column when authorized
`post()`	Send an approved student score
`result()`	Read the student's current LMS result

Every operation obtains a service token for the required scope, validates the destination, applies a timeout, rejects redirects, and parses bounded response data.

Worker walkthrough

`syncOnce(repository, client, job)` follows this sequence:

1. Acquire the job's academic sequencing locks.
2. Check whether a newer grade revision supersedes it.
3. Reuse the stored line-item binding.
4. If no binding exists, discover matching columns.
5. Stop if multiple columns match.
6. Before creating, persist a creation intent.
7. Save the confirmed column binding.
8. Post the exact approved score event.
9. Record `Delivered`.
10. Read results when authorized.
11. Compare normalized values and record `Verified`, `Conflict`, or `VerificationPending`.

The comparison accepts equivalent denominators:

```
Math.abs(
  result.resultScore / resultMaximum -
  payload.scoreGiven / payload.scoreMaximum
) <= 1e-6
```

A result observation is different from a successful score request. Preserve that distinction in the dashboard.

Important retry correction

The earlier worker classified score HTTP 5xx responses as directly retryable. The new policy is more conservative: a write can have been applied before a timeout or server error reaches Semester.

Failure	New handling
Column-create timeout or 5xx	Reconcile creation before another POST
Score timeout or 5xx	`OutcomeUnknown`; reconcile before redelivery
HTTP 429	Retry after server guidance and bounded backoff
HTTP 401	Bounded credential renewal
HTTP 403/404/422	Operator or configuration action
Canvas stale-timestamp 400	Inspect revision order/LMS changes
Result-read timeout or 5xx	Safe bounded read retry
Retry limit reached	Dead-letter review

Do not “fix” a stale score by generating a newer timestamp for the same academic event. That could overwrite a later legitimate change.

Batch updates

The extension batches local jobs—not one bulk AGS request.

```
await dispatchBatch(jobs, deliver, {
  concurrency: 4,
  keyOf: job => job.laneKey
});
```

It processes separate lanes concurrently, orders revisions within a lane, and blocks later work after an unresolved predecessor. A suitable lane includes tenant, deployment, line item, and LTI user.

This local sequencing still needs distributed database ownership. PostgreSQL `FOR UPDATE SKIP LOCKED` supports competing queue consumers, but a row claim alone does not implement revision ordering or lease recovery. [1] [2]

2. `service.mjs` HTTP verification

The service is an internal gateway—not a public browser callback implementation.

Request flow

```
Request arrives
    ↓
Set no-store and JSON response headers
    ↓
Authenticate internal caller
    ↓
Allow POST /verify or POST /mermaid
    ↓
Read bounded request body
    ↓
Parse JSON
    ↓
Invoke verifier or exporter
    ↓
Return authorized context or generic rejection
```

Existing protections

- A sufficiently long service API key is required.
- Authorization comparison uses equal-length constant-time comparison.
- Only two POST routes are accepted.
- Request bodies are limited to 128 KiB.

- Verification errors return a generic rejection.
- Successful responses omit the full private JWT payload.
- Responses use `Cache-Control: no-store`.

For `/verify`, the trusted application supplies:

```
{
  "token": "<id_token>",
  "state": "<callback-state>",
  "browserBinding": "<trusted-gateway-binding>"
}
```

The binding must come from the gateway's established browser correlation. A public client must not choose an arbitrary binding and thereby authenticate itself.

Production additions

Set explicit server timeouts:

```
server.headersTimeout = 10_000;
server.requestTimeout = 15_000;
server.keepAliveTimeout = 5_000;
server.maxHeadersCount = 50;
```

These are proposed values to tune against your proxy configuration. Node documents header and whole-request timeouts as protections against incomplete or slow requests. [3]

Also add:

- Content-type enforcement.
- Rate limits and concurrency caps.
- Managed service credentials or mTLS.
- Credential rotation.
- Strict JSON parsing policy.
- Redacted internal failure codes and trace IDs.
- Tenant-scoped audit events.
- Graceful shutdown.
- Health/readiness endpoints.
- HTTP-layer tests.

The current tests do not exercise the HTTP service.

3. `core.mjs` caching and verification

JWKS cache

`jwksProvider()` works as follows:

```
Lookup registration ID
    ↓
Cached URL matches and five-minute TTL is valid?
    ├── Yes → use cached public keys
    └── No → fetch approved HTTPS JWKS
                  ↓
             reject redirects
                  ↓
             enforce timeout/size limit
                  ↓
             validate key-set shape
                  ↓
             replace cached entry
```

The current cache has a deliberate limitation: it does not immediately refresh on an unknown `kid`. A newly rotated key can therefore fail until cache expiration.

For production, add controlled unknown-key refresh with:

- Concurrent-refresh deduplication.
- A refresh cooldown.
- One verification retry.
- Fail-closed behavior.
- Tested overlap during key rotation.

Avoid a refresh on every invalid signature; that lets malicious tokens generate outbound traffic.

Cryptographic checks

The verifier:

- Accepts only compact RS256 tokens.
- Selects keys only from trusted registration JWKS.
- Requires exactly one compatible key matching `kid`.
- Rejects private key material.
- Rejects unsupported critical headers.
- Requires the proposed minimum RSA key size.
- Verifies the original encoded header/payload bytes.

It never trusts a token-supplied key URL or embedded key.

Claims and authorization

After signature verification, it validates:

- Issuer and audience.
- `azp` when applicable.
- Expiration, issued-at, and optional not-before.
- Transaction nonce.
- LTI version.
- Approved deployment.
- Expected destination.
- Message-specific structures.
- Allowed service and return origins.

Then it invokes authorization and atomically consumes the launch transaction.

The memory store's concurrent-replay test passed, but that only establishes behavior within one process. Multiple application instances require durable atomic compare-and-consume.

Canvas token cache

The service-token provider caches by requested scope set. It reuses sufficiently fresh tokens, deduplicates concurrent requests, and checks effective scopes.

Production cache boundaries should include:

```
Tenant + registration + deployment/credential boundary
+ token audience + scope set + signing-key version
```

Never share tokens across unrelated institutions merely because their scope strings match.

4. Queue and audit architecture

The hardening ZIP contains three table scaffolds:

Table	Purpose
`ags_sync_lanes`	Ordered ownership, lease expiration, fence counter, blocked reason
`ags_sync_jobs`	Immutable sync intent, payload hash, revision, status, attempts, schedule
`ags_sync_audit`	Metadata-only transition history

The schema includes uniqueness constraints and an append-only audit trigger. It has not been applied or tested against PostgreSQL.

Idempotency boundaries

Distinguish two guarantees:

- Local idempotency: repeated enqueue of the same approved revision produces one intent.
- Remote reconciliation: determine what the LMS currently contains after an uncertain request.

Do not claim exactly-once remote delivery.

The enqueue transaction should atomically create the grade-publication event, sync job, and audit record. A duplicate idempotency key must also match the stored payload hash; changed content is not the same intent.

Distributed worker rules

- Claim a lane and its next job atomically.
- Commit the claim before HTTP.
- Renew ownership while processing.
- Require owner/fence checks on local completion writes.
- Reconcile expired `Delivering` jobs before sending again.
- Never assume a local fencing token cancels an already-sent LMS request.

PostgreSQL advisory locks are cooperative application locks; they are not automatically enforced against all writers. [2]

Audit content

Record tenant, job, approved revision, correlation ID, transition, HTTP status, and sanitized reason code.

Do not log bearer tokens, private keys, full JWTs, student feedback, or complete grade payloads.

5. Readiness and executed tests

The new extension passed these seven tests:

- `Retry-After` seconds.
- `Retry-After` HTTP date.
- Uncertain creation classification.
- Ambiguous score 500 classification.
- Rate-limit wait handling.
- Retry exhaustion.
- Same-lane ordering and cross-lane failure isolation.

The earlier reference suite separately passed 16 synthetic/mock tests. These results do not establish production database behavior or Canvas/Blackboard interoperability.

Remaining release gates are durable replay protection, tenant-safe queue repository implementation, atomic audit writes, lease recovery, credential renewal, JWKS rotation, HTTP security tests, outage/load tests, and both LMS sandbox workflows. The attached extension is production-oriented scaffolding—not a production-ready service.

⁂

1. https://www.netdata.cloud/academy/update-skip-locked/
2. https://www.postgresql.org/docs/current/explicit-locking.html
3. https://nodejs.org/api/http.html
4. https://captaincompliance.com/education/tracking-technologies-the-complete-guide-to-adtech-compliance-and-privacy-risk-management/
5. https://tidra.ai/initiatives/nodejs-http-server-timeout/
6. https://news.ycombinator.com/item?id=37647554
7. https://www.matthewswong.com/en/blog/postgres-skip-locked-job-queue/
8. https://www.inferable.ai/blog/posts/postgres-skip-locked
9. https://github.com/nodejs/node/issues/44228
10. https://neon.com/guides/queue-system
11. https://bun.com/reference/node/http/Server/headersTimeout
12. https://nodejs.org/api/https.html
13. https://milanjovanovic.tech/blog/pessimistic-locking-ef-core-postgresql
14. https://www.bigbinary.com/blog/solid-queue
15. https://blog.appsignal.com/2023/11/08/how-to-use-timeouts-in-nodejs.html